

Lineage Collection Technical Discussion Guide

Architecture explanation difficult questions and a focused preparation plan

6 October 2026 | Companion to the architecture review

Baseline: [Kart rc lineage collection at 32958a35cdf49341b15a4d6cb05f596811396b06](#). Current implementation statements refer to this snapshot; proposed examples and targets are labeled. This guide is suitable for a senior technical review and public technical discussion.

The position to defend

Retain the evidence and publication control plane. Make bounded Java/Spring service mappings and Spark/SQL field lineage equally important in a first integrated milestone, connected through Kafka and closed at a downstream service. Use OTel first for services and customized field attributes where feasible, a separate SDK only as fallback, and Spark-native capture. Normalize all lineage into OpenLineage through semantic adapters. Reuse compiler and engine-native evidence, reconcile versions and scopes, and prove useful impact answers on an independent corpus before broadening support or activating LLM inference.

The repository is a substantial prototype with important semantic and integration gaps. The credible discussion is about what each evidence source can establish, which claims are currently implemented, and what test would justify the next claim.

A four minute architecture explanation

This is a speaking script, not a claim that every proposed component is already implemented. At a steady pace it takes about four minutes.

“Our goal is to answer three operational questions: what could break if I change this field, what is affected by this incident, and which upstream change is a plausible cause. A graph is useful only if its edges have clear meaning, refer to the right version of the system, and expose what we do not know.

“I would collect evidence through two first-class paths. For Java/Spring services, analyze a pinned source revision with resolved framework and schema context, then collect independently observed service, database and messaging metadata where the platform allows it. For Spark, use resolved logical plans and supported OpenLineage integration as the primary execution evidence, supplemented by static SQL for pre-deployment impact. Both belong in the first milestone. I would not try to infer arbitrary Spark dataflow by matching method names in Java source.

“All collectors feed a shared typed fact model. It distinguishes service operations, jobs, runs, datasets and fields. An asset identity includes its physical namespace, while a schema version and code artifact identify the version being described. Every assertion retains its extractor version, source citation or execution reference, scope, timestamps and limitations. A field providing a value is different from a field controlling filtering or joining; the graph must retain that distinction.

“The reconciliation layer resolves aliases against a pinned catalog, quarantines ambiguous identities, and keeps static possibilities separate from observed executions. An independent runtime edge missing from static analysis is a useful candidate, not something to discard. An unobserved static edge is not automatically false. Conflicts, collection gaps and unsupported constructs remain visible. We keep immutable evidence, but derive the current graph from evidence valid for the selected artifact and time. Old evidence must not accidentally validate new code.

“Publication is an idempotent, versioned process. The transport can deliver at least once; stable event identities, durable writes and fenced publication prevent duplicate effects and stale publishers. The query returns its graph version, time scope, traversal limit and uncertainty. Impact analysis identifies plausible affected paths. Root-cause analysis needs incident-time failures, deployments and counterevidence in addition to those paths, so it should return ranked hypotheses rather than claim causality from connectivity.

“The current repository has a useful foundation here: exact source snapshots, explicit residue, durable commands, immutable evidence, review and fenced publication. The audit also found important limits. Spark and LLM libraries are not wired into the normal collection path, and SQL and Kafka packs need missing default resolver wiring. Runtime has distinct paths: Python executes actual module functions with injected dataset I/O, Java uses a stubbed harness, and external SDK/OTel/OpenLineage inputs can supply independently observed metadata. Those paths have different strengths. The current pipeline still does not establish independent production transformation verification. We also found incorrect mappings on ordinary SQL and Spark cases, incomplete deletion handling and inconsistent service-node support downstream.

“So my first milestone is a narrow, fully connected service-to-Kafka-to-Spark-to-table-to-consuming-service slice. We prove it through the normal product API, not only a demonstration script. We measure precision, recall, coverage, freshness, overhead and impact-answer quality against independently authored expectations, including missing evidence and adversarial cases. LLMs stay optional and limited to reviewed hypotheses for unresolved gaps. If they cannot show incremental correct coverage at acceptable cost and risk, we leave them out. My review ask is agreement on those semantics, the supported first slice and the evidence required before we call it ready.”

If you get only 30 seconds

“The control plane is worth retaining, but the extraction and integration claims need tightening. I propose one fully tested Java/Spring → Kafka → Spark → table slice, with typed, versioned, provenance-bearing facts and explicit coverage gaps. Runtime must be independent evidence, and LLM output must remain a hypothesis. The slice closes at a downstream service. The decision is whether to fund correctness and the normal end-to-end path before broader connector coverage.”

A whiteboard you can redraw from memory

1. Left: Java/Spring source + database/message metadata; Spark compiled SQL + resolved plans + execution events.
2. Center: typed assertions → qualified identity/schema binding → evidence store → reconciliation, conflicts and gaps.
3. Right: versioned graph → impact, incident blast radius, RCA hypotheses.
4. Below: durable ingestion, idempotency, fences, authorization, retention, telemetry.
5. Above: capability matrix and independent evaluation. Draw a dashed optional LLM box feeding hypotheses, never authoritative publication directly.

Draw external runtime ingestion as a genuine separate input. Python harness execution can also discover observations absent from static analysis; the current reconciliation stage, rather than the observer itself, restricts which observations become graph assertions. Do not draw AWS/EKS as implemented deployment parity; the inspected AWS implementation uses Lambda/ECS/Fargate and still has missing product endpoints.

One running example explain the data before the machinery

Business story

An order service reads a persisted order, publishes an OrderCreated event to Kafka, and a Spark job aggregates valid events into daily sales. A reporting service reads that table and maps revenue_major to its response field dailyRevenue. The question is: “If we rename or change the unit of the order amount, which published metric might be affected, and what evidence supports that answer?”

This is a proposed acceptance scenario and identity contract. It has not been executed by this audit, and the repository does not currently prove this entire chain. The example code revision, schema versions, run IDs, offsets and hashes below are illustrative labels, not measured artifacts.

The narrowly specified computation

- Java/Spring operation: OrderService.publish(orderId) reads orders.total_minor, orders.country and orders.status. An explicit mapper copies them into event fields amount_minor, country and status; event_date is derived from an order timestamp using an explicitly modeled time-zone conversion. For the amount path, the required mapping is orders.total_minor → OrderCreated.amount_minor.
- Kafka binding: the effective production binding resolves to topic orders.created on cluster cluster-eu-1. The event's writer schema is registry subject orders.created-value, version 7, fingerprint schema-event-7. Subject alone is insufficient because it can have many versions.
- Spark decodes with a pinned reader schema compatible with that writer schema, materializes the relation events, and runs the logical equivalent of:

```
INSERT INTO analytics.daily_sales (event_date, country, revenue_major)
SELECT event_date, country,
       SUM(CAST(amount_minor AS DECIMAL(20, 2)) / 100)
FROM events
WHERE status = 'PAID'
GROUP BY event_date, country;
```

The acceptance run processes a frozen Kafka offset interval containing the complete test day and writes one committed output snapshot; the reporting service queries that snapshot. All amounts are declared USD minor units for this example, with a bounded numeric range. This is deliberately a finite run: a continuing streaming implementation must additionally define state, append/update/complete output semantics and idempotent sink commits. It cannot simply rerun INSERT for each microbatch and claim correct daily totals.

The divide-by-100 convention is a declared domain rule for this example. SQL semantics alone do not prove that the input unit really is minor currency units. A code change from cents to another unit can be a semantic breaking change even when the schema stays an integer.

Identities and versions to put on the board

Object	Logical identity in this example	Version/context carried separately
Service operation	service://orders-service/ com.acme.OrderService#publish	Artifact orders-build-A, source commit orders-revision-A, effective deployment/config digest
Source field	urn:ldp:prod:postgres:orders:commerce.pu blic.orders#total_minor	Physical database/cluster ID, schema fingerprint schema-orders-12, field ID if supplied by catalog
Kafka field	urn:ldp:prod:kafka:orders:orders.created# amount_minor	cluster-eu-1, schema subject/version/fingerprint, structured field path \$.amount_minor
Spark operation	Proposed typed job ID spark-job://sales/daily- sales	Artifact sales-build-B, analyzed plan digest plan-P, Spark/connector versions
Spark execution	Proposed typed run ID spark-run://sales/run-R	Attempt ID, execution ID, source offsets, output commit ID, observed time
Target field	urn:ldp:prod:delta:analytics:analytics.daily _sales#revenue_major	Catalog/table identity, schema fingerprint schema-sales-3, output table version snapshot-T
Downstream operation	service://reporting-service/ com.acme.RevenueService#daily	Artifact reporting-build-C, resolved query/mapper and response schema schema-response-2; proposed response FieldVersion for \$.dailyRevenue

The URN examples use the repository's dataset URN shape, but are not an endorsement that this shape alone is sufficient. In particular, current LineageUrn does not supply an explicit cluster/account/region or schema version. Those must be resolved in the proposed physical identity and schema-binding contract; do not silently hide them in a display name. The job/run IDs above are proposed tagged node types, not current implemented URI schemes. The code currently has downstream incompatibilities even for its existing service:// nodes.

What each edge means

1. orders.total_minor → event.amount_minor: **VALUE / copy**, contingent on the mapper/serializer proof. A repository read and a producer send in the same method are not by themselves this proof.
2. event.amount_minor → daily_sales.revenue_major: **VALUE / aggregate**, expression SUM(CAST(amount_minor AS DECIMAL(20,2))/100).
3. event.status → daily_sales output relation: **CONTROL / filter**, predicate status='PAID'. It determines which rows contribute but does not supply the amount value.
4. event.country and event.event_date: direct projected values for their output fields, plus **GROUPING** influence on aggregate membership. Preserve both roles rather than reducing them to one undifferentiated edge.
5. daily_sales.revenue_major → reporting response.dailyRevenue: **VALUE / copy**, only when a supported query, projection and response mapper prove it. A read span supplies weaker interaction evidence.
6. Kafka producer→topic and topic→Spark consumer are **transport/data-movement** facts. Sharing a topic name proves neither exact message correspondence nor payload-field provenance. If event-level correlation is required, carry partition/offset or explicit message identity; structural lineage ordinarily needs no payload values.

For controls affecting many outputs, retain a relation/operation-level dependency and expand only when the query semantics require it. Replicating every filter field against every output field can create large storage overhead and obscure its meaning.

Evidence required to close the path

Pin the service source span, entity/table binding, mapper and serializer mapping, effective Kafka binding and writer/reader schema fingerprints. Add the Spark analyzed plan, run/attempt, bounded input offsets and committed sink snapshot. Then prove the downstream query and response mapper. Independent service/database/message metadata may establish interactions without establishing every field mapping. Each missing segment is an explicit break.

Use three identity layers: a semantic claim family; a claim revision bound to schemas, code and context; and an immutable assertion tied to an extractor and source evidence. Stable source event IDs plus payload digests support idempotent replay. Reusing an event ID with changed content is a conflict. Measure complete source-to-consumer field paths separately from transport connectivity.

Three changes to reason through

- Rename amount_minor to amount_cents: bind a new schema version, inspect writer/reader compatibility and mapper/SQL references, and preserve old history. Similar names do not establish rename continuity.
- Change cents to dollars without changing type or field name: schema compatibility may pass while revenue becomes 100 times wrong. The unit rule needs a domain contract and test; lineage alone cannot prove business meaning.
- Observe no PAID events in one run: record the run context. Absence of a witness does not delete a possible definition-level dependency.

Questions and technical followups

Give the short answer first. Expand only to the evidence needed for the question, and distinguish current behavior from the proposal. The comprehensive report contains the detailed contracts and pinned code references.

1 What are you building that we cannot buy or assemble

“The defensible custom work is evidence reconciliation and the service-to-data gaps specific to our estate. I would reuse parsers and engine instrumentation, and compare a catalog purchase against our actual requirements before building catalog features.”

Followup: SQLGlue is an extraction component; OpenLineage is a contract and integration ecosystem; Marquez is an event/metadata backend; Spline specializes in Spark capture. Collibra/Atlas are broader catalog/governance choices; Monte Carlo/Acceldata emphasize observability.

2 Define lineage precisely What does an edge assert

“An edge is a scoped, versioned dependency assertion: a value dependency, a control dependency or a system interaction. The kind and evidence determine what conclusions a query can draw.”

Followup: For the example, `amount_minor` supplies the aggregate's value; `status` controls inclusion; `country` influences grouping and supplies an output value. Separate possible static relationships from observed executions and declared metadata.

3 Why is parsing insufficient Where is the compiler

“Parsing gives syntax. Correct lineage needs name resolution, scope binding, types and schemas, followed by operator-level dependency propagation.”

Followup: Build lexical/query scopes; bind aliases and qualified names; resolve correlated references against the correct enclosing scope; expand stars against the exact schema; map DML targets by ordinal; propagate through projections, joins, aggregations, unions and windows. Preserve source spans in the IR.

4 Which schema did you bind against and what happens when it changes

“Every assertion must identify the schema and configuration it used, as well as the code artifact. The latest catalog cannot be silently substituted for the historical schema.”

Followup: Separate stable asset identity, immutable schema binding and runtime snapshot/commit identity. Resolve database/schema/table using physical namespace and dialect-specific quote/case rules.

5 How do you get from a Spring repository call to event field lineage

“A table read plus a Kafka send is only an interaction path. Field lineage requires a proof through entity binding, the value expression, the mapper and the serializer.”

Followup: Resolve Spring Data repository/entity/schema chains; trace assignments through a bounded interprocedural slice; understand explicitly supported constructors/mappers; bind serialized field paths including aliases and schema versions. For reflection, dynamic proxies, custom converters and unknown mapping functions, retain dataset/method context and an unresolved field mapping.

6 How do you join across Kafka without inventing connections

“Use the resolved cluster, topic, effective binding and schema version. A shared short topic name, trace ID or consumer-group name is not enough.”

Followup: Distinguish producer and consumer operations, serialization/deserialization, keys versus payload, internal/repartition topics and stream/table joins. For structural dependencies, retain topic/schema bindings; for execution-level correspondence, use producer/consumer records such as partition/offset and run progress, without promising row-level provenance.

7 Why build a Spark source analyzer instead of using the Spark plan

“For executed Spark workloads I would prefer the resolved logical plan. Static analysis still helps pre-deployment review, but it should not guess DataFrame flow from method proximity.”

Followup: Resolved expression IDs disambiguate aliases and self-joins; supported logical-plan visitors propagate field dependencies through operators. Pin Spark/Scala/connector versions, tie the plan to run and artifact, and retain unsupported node coverage.

8 Will you guarantee complete lineage for arbitrary dynamic code

“No. We guarantee explicit supported semantics and honest gaps. Arbitrary dynamic behavior cannot be made universally complete by static analysis, a few test runs or an LLM.”

Followup: Conditional configuration, reflection, generated SQL, external state and UDFs can make exact general analysis infeasible or undecidable. Static analysis can overapproximate potential dependencies for a declared subset; runtime captures executed instances; metadata supplies declared relationships.

9 What SQL cases are most likely to make your answer wrong

“Ambiguous binding and positional semantics are immediate risks, before exotic SQL. The audit already found silent wrong answers, so those become regression gates.”

Followup: Test explicit target-column reorder with `SELECT *`, CTE alias lists, nested scopes and shadowing, correlated subqueries, unqualified join columns, UNION branches, constants, `COUNT(*)`, filters, HAVING, CASE, windows, UPDATE/MERGE, nested fields and dialect differences. A parser accepting a statement is not a proof of its lineage.

10 Does runtime independently discover anything or just replay static claims

“It does execute and can discover new observations. Python runs actual module functions with injected I/O; external SDK/OTel/OpenLineage can bring independent evidence. The important limits are instrumentation, comparison semantics, and existing-edge-only promotion in the supported observation path. Java's stubbed harness is a separate case.”

Followup: On the same source, SCA produced $a \rightarrow b$ while actual Python execution recorded $a \rightarrow b$ and conditional $a \rightarrow c$. CORROBORATED coexisted with one runtime-only observation, but collection emitted only the matched b assertion. The comparison proves supplied endpoint pairs were witnessed, not transform/type/entry-point equivalence. External signed input is a distinct path; the current supported reconciliation still does not admit unmatched observations as new claims.

11 What if runtime sees something static missed or sees nothing

“Keep runtime-only evidence as a separately governed candidate. Keep unobserved static possibilities unless there is a sound reason to retire them. Missing observation is not a negative proof.”

Followup: The supported observation-reconciliation path only promotes corroboration of already admitted edges; it does not mean the Python or external observers cannot discover something else. Change it to admission of trusted observations with explicit source semantics, not automatic publication of every event.

12 What exactly does coverage mean

“We need several denominators: estate coverage, source-scope accounting, semantic support and runtime capture coverage. One green percentage cannot represent all of them.”

Followup: Track discovered versus onboarded workloads; expected versus completed/skipped/failed files; relevant constructs/operations with supported semantics; expected runs/plan nodes/events versus captured/drop/unsupported observations; resolved versus unresolved identity bindings; and question-level answer completeness. A file can be processed while its important operation remains unsupported.

13 How will you prove precision and recall

“Use independently authored, held-out ground truth, score each dependency kind separately, and report unknowns and abstentions alongside precision and recall.”

Followup: Precision = correct asserted edges / asserted edges; recall = correct asserted edges / expected edges for the declared task universe. Fix semantic equivalence, direction, field identity, granularity and version matching before scoring.

14 Where do your confidence numbers come from

“The current bands are policy heuristics, not calibrated probabilities. I would show evidence quality and independence explicitly before displaying any percentage.”

Followup: Current SCA+RUNTIME becomes HIGH and product projection can show VERIFIED/92; that does not mean a measured 92% chance of correctness. Static analysis, source-driven harnesses and LLMs reading the same source can have correlated errors; external engine observations can be independent inputs, but their error dependence and granularity still need assessment.

15 Are your events exactly once

“I would promise at-least-once delivery with idempotent durable effects, not universal exactly-once transport. The guarantee must name the transactional boundary.”

Followup: Assign stable producer/event identities; persist an inbox or unique constraint with accepted payload digest; atomically update local state and outbox where possible; retry after acknowledgement loss; reject identity reuse with different payload. Graph projection can lag and be replayed.

16 How do concurrent collectors late events and deletions reconcile

“Immutable evidence is append-only; the active graph is a deterministic version-and-time-scoped projection. New complete scopes can supersede old assertions, and deletions need explicit evidence.”

Followup: Partition ownership and fencing by tenant/system/scope. Use compare-and-swap on the active graph base/version and deterministic merge rules.

17 How does this survive a billion edges or a hot dataset

“We do not have an enterprise-scale result yet. First reduce unnecessary expansion and writes, then benchmark realistic graph shapes and update rates.”

Followup: Store normalized immutable evidence separately from active edge projections; avoid rewriting the full provenance list per observation. Index adjacency and evidence keys; partition by tenant/system and controlled traversal scope; batch projection updates; keep relation-level controls compact; use incremental snapshots/structural sharing where justified.

18 Does an impact path mean the change will break the consumer

“No. A path identifies possible influence. Breakage also depends on change semantics, operator behavior, contracts and which version is running.”

Followup: A dropped selected field differs from a removed unused source field; an explicit cast or compatible default can absorb a change; a rename can be breaking even if current heuristics call it non-destructive. Report affected paths and known compatibility checks separately from predicted failure.

19 Why use an LLM at all

“Only if it resolves bounded, otherwise unresolved mappings with measurable net benefit. It is optional; deterministic collection and useful answers must work without it.”

Followup: Candidate uses: suggest a mapping explanation for a supported unresolved mapper, propose a catalog alias for review, classify an unsupported pattern, or help an analyst navigate evidence. Let it propose, never fabricate authoritative observations.

20 Do citations and valid URNs prevent LLM hallucinations

“No. A model can cite real text and connect two real fields incorrectly. Syntactic admissibility is not semantic entailment.”

Followup: The current gateway validates nonempty strings, citation substring, dataset-level in-scope membership and proposal count; its resolver is unused. It does not establish that the cited span supports the relation or that the field exists.

21 What about proprietary source PII prompt injection and cost

“Treat source and logs as sensitive, untrusted data. Minimize what is collected and sent, isolate inference, and allow abstention when policy or budgets prevent a safe answer.”

Followup: Prefer schemas, identifiers and plan metadata over row values; scrub literals and secrets; use approved data-region/retention/provider boundaries and per-tenant authorization. Source comments and database text are untrusted instructions, not tool permissions.

22 Are you executing arbitrary repository code safely

“The static path deliberately does not execute repositories. The optional current harness is for trusted non-production sources and is not a sandbox; enterprise execution needs a separate isolated design.”

Followup: Prefer passive platform metadata over running code just to obtain lineage. If active execution is required, use disposable workers with no control-plane credentials, minimized/denied network, read-only inputs, hard process-group timeouts, CPU/memory/output limits, explicit artifacts and authorized dependencies.

23 Can one team learn another team's data model

“Lineage metadata is sensitive. Authorization must apply to collection, evidence, inference, graph traversal and export, not just to the UI landing page.”

Followup: Resolve authenticated identity from the security boundary, not a supplied reviewer name. Partition by tenant and enforce dataset/system policy in queries and cache keys.

24 What is the first milestone and what would make you stop

“One normal-path service-to-Kafka-to-Spark-to-table-to-downstream-service slice, with both service and Spark acceptance gates. Stop expansion if identity, semantics or uncertainty reporting is wrong.”

Followup: Start with explicitly enumerated Spring/JPA/mapping and Spark SQL/DataFrame operators, one Kafka binding/schema mode and chosen database/sink connectors. No application source-code edits by default: repository/catalog metadata and platform agents/listeners/config first; a separate SDK only where OTel customization is infeasible for a required gap.

25 What have you actually proven and what are you asking me to approve

“We have code-level evidence of a strong control-plane prototype and reproducible semantic gaps. We have not proven production capture, enterprise accuracy or scale. I’m asking you to approve the narrow contract and evidence gates, not a readiness claim.”

Followup: The full backend run was 1,286 passed, 59 failed, 34 skipped; all 59 were classified as environment-blocked: 58 Git-trust/ownership failures and one missing-JDK runtime expectation. No security guards were bypassed, so neither call it green nor label these failures product regressions.

Proof plan and preparation

Release evidence to request

These are proposed gates, not measured results or committed delivery dates.

Gate A semantics and identity

- Ratify typed entities, physical identity, schema/artifact/time binding, dependency kinds and observation source classes.
- Independently specify expected assertions for the running example and adversarial variants.
- Make all current silent misbindings fail closed or produce correct edges; no unjustified exact or confidence promotion.
- Include service nodes through serialization/query, qualified table-name collisions and target-column ordering.

Gate B equally weighted service and Spark slice

- Service: exact checkout → entity/schema binding → supported mapper/serializer → effective Kafka binding; independent operation metadata at its actual granularity. Close the chain with output-table query → downstream-service mapper → response/event field.
- Spark: exact artifact/SQL → schema-aware binding plus a real supported engine/OpenLineage capture → sink identity/commit.
- Both: normal product API → durable collection → evidence/review → publication → bounded version-pinned query. Remove script-only assembly from the proof.
- Record every required integration step, whether source edits are needed, and time to first useful result. Missing one side means the slice is incomplete. Require complete evidenced field paths for every declared supported positive reference case and explicit breaks for opaque/negative cases; transport connectivity alone cannot pass.

Gate C truthfulness under change and failure

- Retry/replay/crash/reorder tests at durable boundaries; identical identities cannot produce duplicate effects.
- Artifact/schema changes, deleted files, complete empty scopes, tombstones, late old-version observations and rollback.
- Drops, unsupported nodes, unexecuted branches, unavailable schemas, auth failures and partial runs must become visible incompleteness.
- Compare impact answers across edge orderings and alternative paths. Validate incident blast radius and RCA separately from static impact.

Gate D quality cost and operational acceptance

- Repository-disjoint independent corpus with service and Spark strata; exact precision/recall definitions and confidence intervals. The companion design proposes a one-sided 95% lower precision bound of at least 99%, at least 95% recall within the supported cohort, and at least 90% path completeness on preselected pilot questions. These are proposed gates to validate and negotiate, not measured performance; the reference integration cases still require complete positive paths.
- Report coverage, unresolved/abstained rate, stale-edge rate, false-confidence rate and answer-level completeness, not only aggregate edge scores.
- Realistic graph size, degree skew, updates, history and concurrent queries; measure p95/p99 latency, ingest lag, write amplification, overhead, recovery and cost.
- Tenant authorization, sensitive metadata/LLM boundary and runtime isolation threat review.
- Only evaluate an LLM after the deterministic baseline and abstention workflow are useful; disable it if benefit does not survive the held-out evaluation and cost/risk review.

Three focused preparation sessions

Core preparation is **three 55-minute sessions plus two optional 15-minute refreshers: 2 hours 45 minutes core, 3 hours 15 minutes with refreshers**. Do not try to memorize all 25 answers.

Session 1 explain the architecture 55 minutes

- 0–10: tell the business story without tool names. Explain amount values, filter controls and a schema change.
- 10–25: draw the running example and assign identity, version and evidence to every hop. Ask: “What fact allows me to connect these two fields?”
- 25–40: give the four-minute explanation twice. First use notes; then redraw from memory. Explain the downstream-service response mapping as the final link. Label current implementation versus proposed target as you speak.
- 40–50: answer questions 2–8. For each, predict one counterexample and say what the system should emit.
- 50–55: write three sentences: what we keep, what is blocked, what the reviewer must decide.

Success check: You can explain why a service trace is not automatically field lineage and why Spark plan binding is stronger than method-name matching.

Session 2 adversarial technical questions 55 minutes

- 0–10: reproduce the reasoning for target-column reorder, table-name collision and Spark two-DataFrame misbinding on paper.
- 10–35: have a colleague, or a self-timed voice rehearsal, ask 10 questions drawn from 9–23 in unpredictable order. Answer in 20–30 seconds, then take one follow-up.
- 35–45: defend idempotency, late events, retractions and confidence without saying “exactly once,” “always,” or “guaranteed complete.”
- 45–55: practice a concession: “That is a real gap in the current implementation. The proposed contract addresses it by X; the acceptance test is Y.” Use the specific objection, not a canned apology.

Success check: You can distinguish Python execution, Java stubs and external runtime input, explain capture versus comparison versus promotion, and state a real quality metric with its denominator.

Session 3 mock review and decisions 55 minutes

- 0–5: opening architecture explanation.
- 5–30: uninterrupted mock questions, including “why not buy?,” “what has actually run?” and “what would make you stop?”
- 30–40: walk the proof plan and negotiate one narrower support boundary without dropping either service or Spark from the first slice.
- 40–50: review only the answers that lacked evidence or mixed current and proposed behavior. Replace claims with a source, test or acknowledged unknown.
- 50–55: make a one-page meeting card: opening recommendation; three strengths; five blockers; first-slice gates; three decisions.

Success check: You finish with a clear decision request rather than a tour of modules.

Optional refreshers 15 minutes each

- Before Session 3: rehearse the running example with a filter, a schema rename and an unexecuted branch.
- Before the meeting: say the 30-second and four-minute versions once, then review the evidence limits. Avoid last-minute memorization of vendor feature lists.

Adapt to meeting duration

- **15 minutes:** 3-minute opening, 7-minute challenges, 3-minute proposed gates, 2-minute decisions.
- **30 minutes:** 4-minute opening, 6-minute example, 15-minute challenges, 5-minute decisions.

- **60 minutes:** 5-minute opening, 10-minute example, 30-minute deep dive, 10-minute proof plan, 5-minute decisions.

These are rehearsal shapes, not an assumption about the actual agenda. Ask the organizer about duration and desired decision when convenient.

Questions you should ask the reviewer

1. “Which first decision matters most: safer pre-deployment changes, incident blast radius, or RCA? I propose retaining acceptance for all three, while prioritizing their depth explicitly.”
2. “What false-positive and false-negative costs should determine the first release gates, and who owns independent ground truth?”
3. “Which exact service and Spark versions/operators must the first slice support?”
4. “Can we collect passive production metadata under platform ownership, without application source edits, and what privacy boundaries apply?”
5. “Which pieces should we integrate or buy rather than own, and what benchmark would change that decision?”

Evidence and further reading

Repository grounded evidence

- [Immutable repository baseline](#).
- [Default product wiring](#) and [registered analyzer packs](#).
- [SQL target/projection handling](#), [Spark method-level extraction](#), [schema replay identity](#).
- [Python harness execution/matching](#), [Java runtime matching](#), [generated repository recorder](#).
- [LLM gateway/guards/cache](#), [confidence bands](#), [product score projection](#).
- [Append-all consolidation](#), [local proposal/diff](#), [query traversal](#), [Neptune namespace operations](#).
- [AWS collection integration gap](#).

Component documentation reviewed in the companion market assessment

- [SQLGlott lineage implementation](#): scope/schema-aware extraction machinery.
- [OpenLineage column-lineage facet](#): direct/indirect dependency semantics.
- [OpenLineage Spark column lineage](#): logical-plan-derived evidence and capture boundaries.
- [Spline Spark agent](#): an alternative Spark capture implementation to benchmark.
- [Marquez architecture](#): metadata backend role.

Dynamic documentation describes documented capabilities, not a measured head-to-head result. Pin releases and connector settings before reproducing any benchmark. The companion architecture review provides the full audit, market benchmark, PRD, TDD, and source trail. This guide focuses on the spoken explanation and decisions.

Runtime instrumentation and ATDD questions

The selected proposal is OTel first for services and feasible field-attribute customization, a separate SDK only where that customization is infeasible, Spark-native capture, and OpenLineage as the common lineage contract. A semantic adapter is required between OTel observations and lineage events.

Can we replace static analysis with ATDD runtime capture? No. Runtime witnesses exercised executions; static analysis identifies potential paths, including unexecuted branches. Reconcile them without interpreting non-observation as impossibility.

Is a custom annotation independent verification? Execution of its method is observed; its mapping is an assertion. Independent gold, engine evidence and carefully chosen sink tests assess the assertion. Generating expected edges from the same annotation would be circular.

Why OTel-first but OpenLineage output? OTel supplies service instrumentation, including feasible custom field mappings; Spark supplies engine evidence. A semantic adapter normalizes service observations into OpenLineage. Standard spans alone do not establish field semantics, and conversion does not strengthen a declaration. A separate SDK is only a fallback.

Does 100% ATDD or branch coverage mean complete lineage? No. The scenario catalog, instrumentation scope and gold corpus bound the claim; capture losses and unstitchable boundaries can remain. Report all six measures.

How can fail-open capture support trustworthy verification? Let production work proceed, but make loss observable and downgrade its evidence. In CI, independently expected evidence, durable receipts and bounded completion checks fail the acceptance run when capture is incomplete.

For the concrete ATDD scenario, change source amount, filter behavior and SUM versus MAX while leaving stale annotations intact. Require both independent typed-edge gold and the expected committed sink response. Report scenario, operation, branch, edge, boundary and transport coverage separately. See Part V of the architecture review for the full contracts, code examples and primary sources.